Data Collection and Preparation

Techniques for Data Acquisition

Definition: The process of collecting data from different sources for analysis.

1. Manual Data Collection

- **Surveys & Questionnaires** → customer satisfaction forms, research surveys.
- **Interviews / Focus Groups** → direct interaction to collect qualitative data.
- **Observation** → recording human behavior, events, or processes.

Example: A retail store records customer footfall manually.

Collecting student satisfaction responses via Google Forms.

2. Automated Data Collection

- **IoT Sensors** → temperature, heart rate monitors, smart meters.
- **Log Files** → web server logs, transaction logs.
- **Clickstream Data** → tracking user navigation on websites.

Example: A fitness tracker continuously records heart rate and steps.

3. Web Data Acquisition

- **Web Scraping** → extracting data from websites using tools like BeautifulSoup, Scrapy.
- **APIs (Application Programming Interfaces)** → Twitter API, Google Maps API for structured data.

Example: Pulling tweets with hashtags for sentiment analysis.

4. Database & System Integration

- **SQL/NoSQL Databases** → querying company databases.
- **ETL (Extract, Transform, Load)** → pipeline for combining data from multiple systems.

Example: Banking sector integrates customer transaction data from multiple branches.

5. Open Data Sources

- **Government Portals** (e.g., data.gov.in).
- **Research Repositories** (Kaggle, UCI ML repository).
- **Public APIs** (WHO, World Bank).

Example: Using WHO COVID-19 dataset for prediction models.

6. Crowdsourcing

- Collecting data from a **large group of people online**.
- **Platforms:** Amazon Mechanical Turk, Google Forms. Example: Collecting image labels for AI training.

Handling Missing Data and Outliers

Problem: Data is often incomplete or contains extreme values.

1. Identify Missing Data

- Represented as `NaN`, blanks, NULLs.
- Methods: `.isnull()` in Pandas, summary statistics, or visualization (heatmaps).

2. Techniques to Handle Missing Data

I. Deletion Methods

a) **Listwise deletion** → remove entire row with missing values.

b) **Column deletion** → drop column if too many missing values.

Best if dataset is large and missing % is small.

II. Imputation Methods

a) **Mean / Median / Mode Imputation**

- Numeric: replace with mean/median.
- Categorical: replace with mode.

b) **Forward / Backward Fill** → fill using nearby values (time series).

c) **Regression / KNN Imputation** → predict missing values based on other variables.

III. Flagging

- Create a new feature (e.g., is_missing = 1/0).

  Useful when missingness itself carries information (e.g., loan defaults).

Outlier Handling Approaches

1. Identify Outliers

- **Statistical methods**:
  - Z-score > 3 or < -3 → potential outlier.
  - IQR method: Values < Q1 – 1.5×IQR or > Q3 + 1.5×IQR.
- **Visualization**:
  - Box plots, scatter plots, histograms.

2. Techniques to Handle Outliers

1.Remove Outliers

Drop rows if they are clearly erroneous (e.g., negative age).

2.Transformation

Apply **log or square root** to reduce skewness.

3.Capping

Replace extreme values with nearest threshold (e.g., cap income at 99th percentile).

4.Imputation

Replace with mean/median if value is too far from distribution.

5.Keep Them

If they represent rare but valid cases (e.g., fraud detection, billionaires in income data)

🔷 Example

Dataset: Employee salaries (₹)

```csharp
[25,000, 28,000, 30,000, 32,000, 35,000, 90,00,000]
```

- **Outlier** = 90,00,000 (much higher).
- Handling:
  - **Remove** (if entry error).
  - **Cap** at 95th percentile (~40,000).
  - **Log Transform** → log10(90,00,000) ≈ 6.95, compressing extreme

🔹 **What's happening here?**

- Most employees earn around **₹25,000 – ₹35,000**.
- But one employee (or one record) shows **₹90,00,000** → this value is **extremely far away** from the rest. This is called an **outlier**.

🔹 **Why is it a problem?**

That's misleading because the "average" salary looks **₹15.25 lakhs**, while most people actually earn ~₹30,000.
Models may think salaries vary wildly.
Visualizations (like histograms) get stretched.

Data Scrubbing and Cleaning

Definition: Correcting errors and inconsistencies in raw data.

Common Cleaning Steps with Examples:

1. **Removing duplicates:** If a student appears twice in the dataset.
2. **Correcting typos/inconsistencies:** “ECE” vs. “EcE” vs. “ece” → standardize to “ECE”.
3. **Validating ranges:** Age cannot be negative, Score must be ≤100.
4. **Formatting standardization:** Dates as `YYYY-MM-DD` instead of mixed formats.

Example: In survey data, one record shows *Department = “123”* (invalid). Data cleaning replaces it with a correct category or flags it as invalid.

Data Transformation and Normalization

Definition: Converting raw data into a more usable and uniform format.

1. **Scaling/Normalization:** Adjusting values to a common scale.
   I. **Min–Max normalization:** It's a **data scaling technique** that **rescales values into a fixed range**, usually **0 to 1**.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Where:

- x = original value
- $\min(x)$ = minimum value in the column
- $\max(x)$ = maximum value in the column
- x' = normalized value (between 0 and 1)

Example: Scores [66, 92, 70] → normalized as [0, 1, 0.14].

🔷 Example

Suppose we have **Scores**:

```csharp
[66, 70, 74, 78, 80, 81, 85, 88, 90, 92]
```

- Minimum = 66
- Maximum = 92

Now normalize:

For Score = 66:

$$x' = \frac{66 - 66}{92 - 66} = 0$$

For Score = 92:

$$x' = \frac{92 - 66}{92 - 66} = 1$$

For Score = 81:

$$x' = \frac{81 - 66}{26} = \frac{15}{26} \approx 0.58$$

👉 Normalized dataset (approx):

```csharp
[0.00, 0.15, 0.31, 0.46, 0.54, 0.58, 0.73, 0.85, 0.92, 1.00]
```

II. Z-score standardization:

It's a **data normalization technique** that rescales numeric data so that:

- The **mean (μ)** becomes `0`
- The **standard deviation (σ)** becomes `1`

Formula: $(x - \mu)/\sigma \rightarrow$ mean = 0, std dev = 1.

Why Use Z-score Standardization?

- Many ML algorithms (like Logistic Regression, SVM, KNN, Neural Networks) perform better when features are on the **same scale**.
- Helps identify **outliers** (values with |z| > 3 are usually considered outliers).

🔷 Example

Suppose we have student **Scores**:

```csharp
[66, 70, 74, 78, 80, 81, 85, 88, 90, 92]
```

1. Mean (μ) = 80.4
2. Standard Deviation (σ) ≈ 8.5

Now, for Score = 92:

$$z = \frac{92 - 80.4}{8.5} \approx 1.36$$

For Score = 66:

$$z = \frac{66 - 80.4}{8.5} \approx -1.69$$

👉 So, after standardization:

- 92 → +1.36 (above average)
- 66 → -1.69 (below average)

2. Encoding categorical data:

I. ***Label encoding:*** Label Encoding is a **data preprocessing technique** where **categorical values are converted into numeric values** by assigning a unique integer to each category.

Why Use It?

It's simple and memory efficient.

Works well when categories have **natural order** (ordinal data).

Example: Ratings → Low=0, Medium=1, High=2.

Where It's Useful

When categories are ordinal (ordered).

Example: T-shirt sizes → Small=0, Medium=1, Large=2.

When using tree-based models (Decision Trees, Random Forests, XGBoost), which handle label-encoded features well without assuming order.

🔷 Example

Suppose we have the same column **Department**:

```vbnet

Department
CSE
ECE
ME
CSE
ME
```

After Label Encoding:

Department	Encoded
CSE	0
ECE	1
ME	2
CSE	0
ME	2

👉 Here:

- CSE = 0
- ECE = 1
- ME = 2

II. One-hot encoding: One-hot encoding is a **data transformation technique** used to convert **categorical data** (non-numeric labels) into a **numeric format** that can be used by machine learning models.

Example: Suppose we have a column **Department** with 3 categories:

Department
CSE
ECE
ME
CSE
ME

CSE	ECE	ME
1	0	0
0	1	0
0	0	1
1	0	0
0	0	1

Here:

- If "CSE", then vector = [1, 0, 0]
- If "ECE", then vector = [0, 1, 0]
- If "ME", then vector = [0, 0, 1]

One-hot encoding = turning categories into binary vectors, so algorithms can understand them without implying order.

Where It's Used

- Preprocessing categorical features before feeding into ML algorithms like Logistic Regression, Neural Networks, or Decision Trees.
- NLP tasks where words can be encoded as one-hot vectors before embeddings.

3. Log transformation: Log transformation is a **data transformation technique** where we apply a logarithmic function (like log base 10, natural log ln, or log base 2) to each data value.

It's mainly used to **reduce skewness** and **stabilize variance** in data.

Example: Transforming skewed income data to reduce variance.

$$x' = \log(x + c)$$

(where c is a constant added if data has zeros, since $\log(0)$ is undefined).

Why Use It?

Many real-world datasets (e.g., income, population, sales) are **skewed** (long tail).

Log transformation makes distributions more **normal-shaped**, which helps statistical tests and machine learning models.

Reduces the effect of **extreme values/outliers**.

🔷 Example

Suppose we have salaries of employees (in ₹):

```
csharp

[20,000, 25,000, 30,000, 50,000, 1,00,000, 5,00,000]
```

Clearly **skewed** (one very high value dominates).

Now apply **log base 10**:

- log10(20,000) ≈ 4.3
- log10(25,000) ≈ 4.4
- log10(30,000) ≈ 4.5
- log10(50,000) ≈ 4.7
- log10(1,00,000) = 5
- log10(5,00,000) ≈ 5.7

👉 Transformed data:

```
csharp

[4.3, 4.4, 4.5, 4.7, 5.0, 5.7]
```

Notice: The range shrinks, outliers are **pulled closer**, and distribution is more balanced.

Where It's Used

- **Financial data** (income, stock prices).
- **Population data** (city sizes, word frequency in NLP).
- **Machine learning preprocessing** (to make data closer to normal distribution).

4. Binning: Grouping continuous values into categories.

Binning (or **discretization**) is a technique of **converting continuous data into categories (bins)**.

- It reduces the effect of **minor observation errors** or **small fluctuations**.
- Helps simplify data and highlight broader trends.

Types of Binning

1. **Equal-width binning**
    1. Divide the range into intervals of equal size.
    2. Example: If *Scores* range from 0–100 and you create 5 bins, each bin covers 20 points:

    [0–20], [21–40], [41–60], [61–80], [81–100].
2. **Equal-frequency binning**
    1. Divide data so each bin has (approximately) the same number of observations.
    2. Example: If you have 100 students and want 5 bins → 20 students per bin.
4. **Custom binning**
    1. Define bins based on domain knowledge.
    2. Example: Age groups → *Young (18–25), Adult (26–40), Senior (41+)*.

🔷 Example

Suppose we have student **Scores**:

```csharp
[66, 70, 74, 78, 80, 81, 85, 88, 90, 92]
```

Equal-width binning (3 bins):

- Range = 92 – 66 = 26
- Bin size = 26 ÷ 3 ≈ 8.7
- Bins → [66–74.7], [74.7–83.4], [83.4–92]

Result:

- Bin 1: [66, 70, 74]
- Bin 2: [78, 80, 81]
- Bin 3: [85, 88, 90, 92]

Equal-frequency binning (3 bins):

- 10 values ÷ 3 ≈ 3 per bin.
- Bin 1: [66, 70, 74]
- Bin 2: [78, 80, 81]
- Bin 3: [85, 88, 90, 92]

  (Same grouping here by coincidence).

Why Use Binning?

- To handle **noisy data** by smoothing values.
- To make patterns easier to see.
- Useful in decision trees (where splits are categorical).